

Distributed/Parallel AI Processing

Professor Hyeonsang Eom (엄현상 교수)

Distributed Computing Systems Laboratory
Department of Computer Science and Engineering
Seoul National University, Korea

September 28, 2026
Distributed Systems, Fall 2026

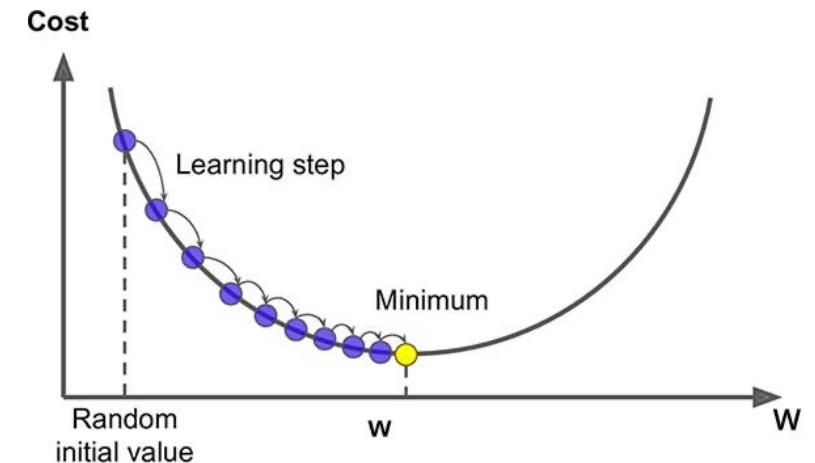
Table of Contents

- ❖ Background
- ❖ Data Parallelism for Gradient Synchronization
- ❖ Pipeline Parallelism with Activation Transfer
- ❖ Tensor Parallelism
- ❖ Comparison
- ❖ Summary

Background

❖ Gradient Descent

- Foundational optimization algorithm
 - Training deep learning models by iteratively minimizing a loss function
 - Adjusting a neural network's weights and biases so the model can learn from its mistakes and improve its prediction accuracy over time
 - 1. **Calculate the Gradients:** using the Backpropagation Algorithm, the network calculates the partial derivatives of the loss function, which determines the gradient - a vector pointing in the direction of the steepest ascent
 - 2. **Update the Parameters:** the algorithm modifies the weights and biases by shifting them in the opposite direction of the gradient (the steepest descent)



Data Parallelism for Gradient Synchronization

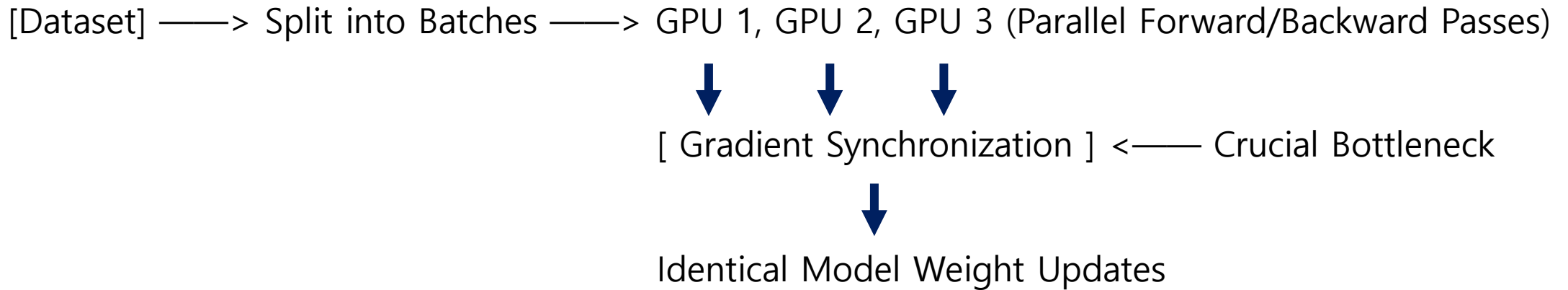
❖ Data Parallelism

- Distributed training technique where a large dataset is split into smaller chunks, and each chunk is processed simultaneously across multiple computing nodes (GPUs)
 - Data parallelism replicates the exact same model across all GPUs
 - Each GPU computes gradients on its own unique slice of data

❖ Gradient Synchronization

- Critical step where these independent gradients are averaged across all nodes to ensure every GPU updates its weights identically

Training a Model with Data Parallelism



- ❖ Replication: The complete model weights are copied to every GPU node
- ❖ Scatter (Data Partitioning): A global mini-batch of data is split evenly
- ❖ Local Forward & Backward Pass: Each GPU processes its local batch independently
- ❖ Synchronization (The All-Reduce Step)
- ❖ Weight Update

Core Communication Architectures

- ❖ Parameter Server (PS), Creating a Severe Network Bottleneck
 - Dedicated "Server" nodes
 - Collect gradients from worker nodes
 - Calculate the average
 - Push the updated weights back to the workers
- ❖ Ring – AllReduce, Highly Scalable
 - Nodes arranged in a logical ring topology
 - E.g., PyTorch Distributed Data Parallel (DDP)

Pipeline Parallelism with Activation Transfer

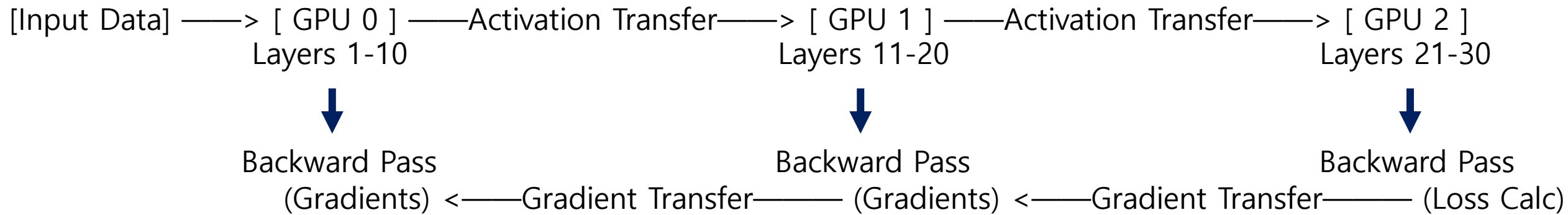
❖ Pipeline Parallelism

- Distributed training technique used when a deep learning model is too large to fit into the memory of a single GPU
 - Slicing the model **layer by layer** across a linear chain of GPUs

❖ Activation Transfer

- Critical communication step where one GPU finishes processing its assigned layers and passes its output activations forward to the next GPU in the pipeline

Training a Model with Pipeline Parallelism



❖ Forward Pass & Transfer

- GPU 0 processes the initial layers (e.g., layers 1–10) using the input data batch
- Output tensors of layer 10 (**activations**) are then transferred across the network interconnect (like NVLink) to the memory of GPU 1

❖ Sequential Execution: GPU 1 sits idle until it receives those activations

Pipeline Bubble

❖ Activation Stash: During this forward pass, each GPU must retain its calculated activations in its local memory

Tensor Parallelism

❖ Tensor

- Multi-dimensional arrays of numbers (weights and activations) that make up the internal mathematical operations of a neural network layer

❖ Tensor Parallelism

- Splicing a single mathematical operation (like a matrix multiplication) across multiple GPUs
 - Tensor itself is broken into shards, allowing multiple GPUs to calculate parts of the exact same layer simultaneously

How Tensors are Split: Megatron-LM Architecture

- ❖ In modern LLMs, the core workload consists of Matrix Multiplications ($Y = X \cdot W$) inside Transformer Attention and Feed-Forward Network (FFN) blocks
- ❖ Tensor Parallelism splits these weight tensors (W) across GPUs in two primary ways: **Column-parallel** and **Row-parallel**

- These two methods are chained together in pair to avoid massive communication bottlenecks

- Column-Parallel Linear Layer (Slicing the First Tensor)

$$(X) \times [W_1 \mid W_2] \text{ (Weight Tensor split by columns)} = [Y_1 \mid Y_2] \text{ (Split Output Tensor)}$$

- Row-Parallel Linear Layer (Slicing the Second Tensor)

$$[Y_1] \times [W_{\text{row1}}] + [Y_2] \times [W_{\text{row2}}] \text{ ==All-Reduce==> Complete Output Tensor (Z)}$$

Comparison of Parallelism Types

Parallelism Type	What gets split?	Granularity Level	Primary Bottleneck
Data Parallelism	The input training dataset batches	High (Batch level)	Gradient synchronization (All-Reduce)
Pipeline Parallelism	The layers of the model network	Medium (Layer level)	Idle hardware time (The "Pipeline Bubble")
Tensor Parallelism	The intra-layer weight and activation matrices	Low (Operation level)	Inter-GPU network latency (Requiring ultra-fast NVLink)

Summary

- ❖ **Data Parallelism**
 - SIMD
- ❖ **Pipeline Parallelism**
- ❖ **Tensor Parallelism**

Thank you!

엄현상 교수
(hseom@snu.ac.kr)